Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные Науки»

КАФЕДРА ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Сжатие файла методом Хаффмана

Студент: _____ Подобедов В. В.

дата, подпись Ф.И.О.

Преподаватель: _____ Волкова Л. Л.

дата, подпись Ф.И.О.

Москва, 2023

Содержание

Введение **3**

1 Аналитическая часть **4**

2 Конструкторская часть **5**

2.1 Создание дерева Хаффмана 5

2.2 Кодирование элементов 5

2.3 Вычисление коэффициента сжатия 6

3 Технологическая часть **7**

4 Исследовательская часть **11**

Заключение **25**

Список используемых источников **26**

Введение

Цель лабораторной работы: реализовать алгоритмы сжатия по Хаффману.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать сжатие по Хаффману.
2. Реализовать сжатие файла.
3. Реализовать возможность считывать данные из файла.
4. Реализовать возможность записать ответ в бинарный файл.
5. Выполнить тестирование реализации разработанного алгоритма.

Согласно варианту, требуется разработать сжатие файла по Хаффману для исходного кода программы на LISP.

1 Аналитическая часть

Сжатие файла по алгоритму Хаффмана [1] — это метод кодирования данных, который использует переменные длины кодовых слов для представления символов в исходном тексте. Он основан на идее присвоения более коротких кодов более часто встречающимся символам и более длинных кодов менее часто встречающимся символам.

LISP — это функциональный язык программирования, который известен своим использованием списков как основной структуры данных и кода. Файлы LISP могут содержать определения функций, переменных, структур данных, а также вызовы функций, операции и другие выражения на языке LISP.

2 Конструкторская часть

2.1 Создание дерева Хаффмана

1. Подсчет частот символов.
   - Создать пустой словарь (map) “freq”, где будут храниться частоты символов.
   - Пройтись по всем символам в тексте и увеличить частоту символа в словаре “freq”.
2. Создание приоритетной очереди.
   - Приоритетная очередь “pq” используется для хранения узлов дерева.
   - Для каждой буквы из словаря “freq” создать узел и поместить его в приоритетную очередь “pq”.
3. Построение дерева Хаффмана.
   - Выполнять нижеперечисленные действия, пока в приоритетной очереди не останется только один узел.
   - Извлечь два узла с минимальными частотами (две самые редкие буквы). Создать новый узел с суммой частот этих двух узлов и добавить обратно в очередь.
4. Получение корня дерева.
   - Корень дерева Хаффмана остается в приоритетной очереди после всех итераций. Извлечь его из очереди и присвоить его корню дерева Хаффмана.

2.2 Кодирование элементов

1. Функция “encode” заполняет словарь “huffmanCode” кодами Хаффмана для каждой буквы.
2. Параметры функции.
   - “Node* root” — указатель на текущий узел дерева Хаффмана, начиная с корня.
   - “string str” — строка, содержащая текущий код Хаффмана для данного узла.
   - “map<char, string> &huffmanCode”’ — словарь, в который мы записываем коды Хаффмана для символов.
3. Проверка наличия узла.
   - Если текущий узел “root” равен “nullptr”, то выйти из функции, так как достигнут конец ветви дерева.
4. Проверка листовых узлов.

— Провеить, является ли текущий узел листовым, то есть у него нет потомков. Если это лист, записать текущий код “str” в словарь “huffmanCode” по ключу “root->ch”.

5. Рекурсивный обход дерева.

   — Рекурсивно вызвать “encode” для левого потомка текущего узла, добавить к текущему коду символа "0".

   — Рекурсивно вызвать “encode” для правого потомка текущего узла, добавить к текущему коду символа "1".

2.3 Вычисление коэффициента сжатия

1. Подготовка переменных.

   — “int bit_1 = text.size() * 8” рассчитать общее количество битов в исходном тексте. Умножение на 8 осуществляется, потому что каждый символ представляется 8 битами в стандартной кодировке ASCII.

   — “int bit_2 = 0” эта переменная будет использоваться для подсчета общего количества битов в закодированном тексте.

2. Подсчет битов закодированного текста.

   — Цикл “for” проходит по каждой паре в словаре “huffmanCode”, который содержит символы и их соответствующие коды Хаффмана.

   — Для каждой пары вычислить количество битов, занимаемых кодом Хаффмана для символа “pair.second.size()” и умножить на частоту символа в исходном тексте “freq.at(pair.first)”. Это происходит потому, что сжатие зависит от того, сколько битов занимает каждый символ после кодирования и как часто он встречается в исходном тексте. Полученные значения присвоить переменной “bit_2”.

3. Вычисление коэффициента сжатия.

   — “(bit_1 + 0.0) / bit_2” Рассчитывает отношение количества битов исходного текста к количеству битов закодированного текста. Используется тип double, чтобы сохранить десятичную часть коэффициента.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки <iostream>, которая предоставляет возможность пользоваься средствами ввода-вывода для стандартной консоли, были подключена библиотека <queue>. Она представляет собой контейнер, который работает по принципу FIFO и обладает собственным набором функций. Была подключена библиотека <map> — контейнер, представляющий отображение ключей на значения. Библиотека <vector> — это динамический массив, который обеспечивает быстрый доступ к элементам и динамическое изменение размера.Также была подключена библиотека <string> для пользования строками. Был также написан стек на ссылках. Для чтения и записи данных из/в файл была подключена библиотека <fstream>.

Далее приведен листинг реализации стека (листинг 1).

Листинг 1 – Исходный код программы для сжатия по Хаффману

```
#include <iostream>
#include <fstream>
#include <queue>
#include <map>
#include <vector>
#include <string>

using namespace std;

struct Node {
        char ch;
        int freq;
        Node *left, *right;
};

struct Compare {
        bool operator()(Node* l, Node* r) {
                return (l->freq > r->freq);
        }
};

Node* getNode(char ch, int freq, Node* left, Node* right) {
        Node* node = new Node();
        node->ch = ch;
        node->freq = freq;
        node->left = left;
        node->right = right;
        return node;
```

```
}

void encode(Node* root, string str, map<char, string> &huffmanCode) {
        if (root == nullptr) return;
        if (!root->left && !root->right) {
                huffmanCode[root->ch] = str;
        }
        encode(root->left, str + "0", huffmanCode);
        encode(root->right, str + "1", huffmanCode);
}

void writeHuffmanCodesToFile(const map<char, string>& huffmanCode, const
   string& encodedText, const map<char, int>& freq) {
        ofstream outputFile("C:\\Users\\Kononova\\Desktop\\huffman_codes.
           bin", ios::binary);

        if (outputFile.is_open()) {
                for (const auto& pair : huffmanCode) {
                        outputFile.put(pair.first);

                        int frequency = freq.at(pair.first);
                        outputFile.write(reinterpret_cast<const char*>(&
                           frequency), sizeof(frequency));

                        char codeLength = static_cast<char>(pair.second.
                           length());
                        outputFile.put(codeLength);
                }

                for (char ch : encodedText) {
                        outputFile.put(ch);
                }

                outputFile.close();
                cout << "Данные успешнозаписанывбинарныйфайл      '
                   huffman_codes.bin'" << endl;
        }
        else {
                cout << "Невозможно открытьфайлдлязаписи   " << endl;
        }
}
```

```
void winnings(string text, const map<char, string>& huffmanCode, const
   map<char, int>& freq) {
        int bit_1 = text.size() * 8;
        int bit_2 = 0;

        for (auto &pair : huffmanCode) {
                bit_2 += pair.second.size() *  freq.at(pair.first);
        }
        double win = (bit_1 + 0.0) / bit_2;
        cout << "Сжатие в " << win << " раз" << endl;
}

void buildHuffmanTree(string text) {
        map<char, int> freq;
        for (char ch : text) {
                freq[ch]++;
        }

        priority_queue<Node*, vector<Node*>, Compare> pq;

        for (auto pair : freq) {
                pq.push(getNode(pair.first, pair.second, nullptr, nullptr
                   ));
        }

        while (pq.size() != 1) {
                Node* left = pq.top(); pq.pop();
                Node* right = pq.top(); pq.pop();

                int sumFreq = left->freq + right->freq;
                pq.push(getNode('\0', sumFreq, left, right));
        }

        Node* root = pq.top();

        map<char, string> huffmanCode;
        encode(root, "", huffmanCode);

        string encodedText;
        for (char ch : text) {
                encodedText += huffmanCode[ch];
        }
```

```
        cout << "Частоты букв: " << endl;
        for (auto pair : freq) {
                cout << pair.first << " : " << pair.second << endl;
        }
        cout << endl;

        cout << "Код букв: " << endl;
        for (auto pair : huffmanCode) {
                cout << pair.first << " : " << pair.second << endl;
        }
        cout << endl;

        cout << "Закодированный текстпоХаффману  : " << encodedText << endl;

        writeHuffmanCodesToFile(huffmanCode, encodedText, freq);
        cout << endl;
        winnings(text, huffmanCode, freq);
}

int main() {
        setlocale(0, "");
        ifstream file("C:\\Users\\Kononova\\Desktop\\huff.lisp");
        string text = "";
        if (file.is_open()) {
                string text_help = "";
                while (getline(file, text_help)){
                        text += text_help;
                        text_help = "";
                }
                file.close();
        }
        else {
                cout << "Невозможно открытьфайл ";
                return 0;
        }
        cout << "Прочитанная фраза: " <<text << endl;
        buildHuffmanTree(text);

        return 0;
}
```

4 Исследовательская часть

Далее на рисунках представлен результат работы программы. Размер входного файла с каждым тестом уменьшается на 20 %.

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 1.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)        (t (+ (fibonacci
 (- n 1)) (fibonacci (- n 2))))))(format t "Числа Фибоначчи: ")(dotimes (i 10)  (format t "~a " (fibo
nacci i)))
Частоты букв:
Ф : 1
Ч : 1
а : 2
б : 1
и : 3
л : 1
н : 1
о : 1
с : 1
ч : 2
  : 49
" : 4
( : 18
) : 18
+ : 1
- : 2
0 : 3
1 : 4
2 : 1
: : 1
= : 2
a : 7
b : 4
c : 9
d : 3
e : 2
f : 7
i : 11
m : 3
n : 11
o : 8
r : 2
s : 1
t : 6
u : 1
~ : 1
```

Рис. 1 – Пример №1

```
Код букв:
ф : 11010010
ч : 11101110
а : 0111011
б : 0110000
и : 011011
л : 110100011
н : 111011100
о : 110110100
с : 0110001
ч : 1110000
  : 10
" : 111001
( : 000
) : 001
+ : 111011101
- : 1101110
0 : 011100
1 : 111010
2 : 110110000
: : 111011111
= : 1110001
a : 11000
b : 110101
c : 11111
d : 011010
e : 11010000
f : 11001
i : 0100
m : 011001
n : 0101
o : 11110
r : 1101111
s : 0111010
t : 01111
u : 110110111
~ : 110110001

Закодированный текст по Хаффману: 0000110101101000110011101101101011011001010011010111110010111000111
1111111101001000001010011010000111111111100101011010100000001110001100101100111000011001110000110101010
1010101000000001110001100101110111010001101110100011010101010101010100000111110000111011011000011001010001
1010111111001011100011111111111010010000011011101001011011101000100110000110010100110101111110010111000011
1111111110100100001101110100101101101100000100100100100100100100011001111101101111011001110000111110011111
1011100111101110011011011000111010011011101110110110100100110110110000011011010111011000111011111000011110
0000110111110111110111001001000011010111100111110100011001110100001110101000000100101110100111000011010
0001100111111011011110110011110000111110011111011100111011001110001011100110000110010100110101111100101
110001111111111101001001000010010001
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 1.91089 раз
```

Рис. 2 – Пример №1 (продолжение)

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 3.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)        (t (+ (fibonacci
 (- n 1)) (fibonacci (- n 2))))))(format t "Числа Фибоначчи: ")(dotimes (i 10)  (format t "~a "
Частоты букв:
Ф : 1
Ч : 1
а : 2
б : 1
и : 3
л : 1
н : 1
о : 1
с : 1
ч : 2
  : 47
" : 4
( : 17
) : 15
+ : 1
- : 2
0 : 3
1 : 4
2 : 1
: : 1
= : 2
a : 6
b : 3
c : 7
d : 3
e : 2
f : 6
i : 8
m : 3
n : 10
o : 7
r : 2
s : 1
t : 6
u : 1
~ : 1
```

Рис. 3 – Пример №2

```
Код букв:
Ф : 0100110
Ч : 0001110
а : 1100111
б : 0110000
и : 110100
л : 0100111
н : 0001100
о : 11010110
с : 0110001
ч : 000000
  : 10
" : 00001
( : 001
) : 1110
+ : 0001101
- : 010000
0 : 110010
1 : 00010
2 : 11010100
: : 0001111
= : 000001
a : 11000
b : 011011
c : 11011
d : 011010
e : 010010
f : 01111
i : 11111
m : 011001
n : 0101
o : 11110
r : 010001
s : 1100110
t : 01110
u : 11010111
~ : 11010101

Закодированный текст по Хаффману: 0010110100100100111111010111010110011111111101101111110010111000110
1111011111111000101011110101000111011111110010101101010001001000000110010110110010111010110010111010101
0101010101000100100000110010110000101110100001011101010101010101010001011101000100011011000101111111
1011011111100101110001101111011111111000101000010010110000101110111010001011111111101101111110010111 0
0011011110111111111000101000010010110110101001110111011101110111000101111111100100010110011100000 11
1010011101000000100011101101000110001010011111001111001001101101000110000110101100001100110011100000000
0000011010000001111100000111100010110101111001110111110110010100101100110100011111111000001011001011101 0
1000101111111100100010110011100001110100111010000011101010111000100000 1
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 1.90323 раз
```

Рис. 4 – Пример №2 (продолжение)

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 5.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)        (t (+ (fibonacci
 (- n 1)) (fibonacci (- n 2))))))(format t "Числа Фибоначчи: ")(dotimes (i 10)
Частоты букв:
Ф : 1
Ч : 1
а : 2
б : 1
и : 3
л : 1
н : 1
о : 1
с : 1
ч : 2
  : 42
" : 2
( : 16
) : 15
+ : 1
- : 2
0 : 3
1 : 4
2 : 1
: : 1
= : 2
a : 4
b : 3
c : 7
d : 3
e : 2
f : 5
i : 8
m : 2
n : 10
o : 6
r : 1
s : 1
t : 4
u : 1
```

Рис. 5 – Пример №3

```
Код букв:
Ф : 0011000
Ч : 0001100
а : 000101
б : 0011011
и : 110101
л : 0011001
н : 0110100
о : 0001000
с : 0001110
ч : 000010
  : 10
" : 001111
( : 010
) : 1111
+ : 0110101
- : 011011
0 : 111010
1 : 00000
2 : 0001111
: : 0001101
= : 000011
a : 01100
b : 1101100
c : 11100
d : 110011
e : 001110
f : 11000
i : 0010
m : 1100101
n : 0111
o : 11011
r : 1100100
s : 0011010
t : 111011
u : 0001001

Закодированный текст по Хаффману: 0101100110011101100000010010111101100000101101001101101110110011100
1110000101001001111111101001011100110110111110011100100100000111001111011101011111011101011111010101
1010101001001000001110011111000000111111000000011111010101010101010100101110111001001101011001011000001011
0100110110111011001110011100001010010011011100111110000001111111111001011000000101101001101101110110011
1
0011100001010010011011100111110000111111111111111111111111111111110101100011011110010011001010110011101110
1110111000111100011001101010001110001100100010110001100011010100110110001000011010000010100001000000010
1101010001101100011111111101011001111011111011001011001010011100011010100010001010000001110101111
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 1.91904 раз
```

Рис. 6 – Пример №3 (продолжение)

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 7.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)        (t (+ (fibonacci
 (- n 1)) (fibonacci (- n 2))))))(format t "Числа Фибоначчи: ")
Частоты букв:
Ф : 1
Ч : 1
а : 2
б : 1
и : 3
л : 1
н : 1
о : 1
с : 1
ч : 2
  : 40
" : 2
( : 14
) : 14
+ : 1
- : 2
0 : 2
1 : 3
2 : 1
: : 1
= : 2
a : 4
b : 3
c : 7
d : 2
e : 1
f : 5
i : 6
m : 1
n : 10
o : 5
r : 1
t : 3
u : 1
```

Рис. 7 – Пример №4

```
Код букв:
ф : 0110010
ч : 0100110
а : 011111
б : 0111010
и : 111011
л : 0110011
н : 0111000
о : 0101111
с : 0111011
ч : 010101
  : 10
" : 010100
( : 001
) : 000
+ : 0111001
- : 010000
0 : 011000
1 : 111000
2 : 0101110
: : 0100111
= : 010110
a : 01101
b : 111010
c : 11111
d : 010001
e : 0100101
f : 11000
i : 11110
m : 0100100
n : 1101
o : 11001
r : 0111101
t : 1111001
u : 0111100

Закодированный текст по Хаффману: 00101000101001011100001111001101101100011110111101011001110101101111
1111111111110100011101000101000111111111001110101000110001001010110101101100110000001001100000001010101 0
1010101000100101011010110110111000000010111000000010101010101010100011110011000101110011000111000111101
1101011001110101101111111111111111010001010000101101101110000000000100011100011111011101011001110101101 1
1111111111111010001010000101101100101110000000000000000000000011100011001011110101001000110111100110111
0011001010000100110111011011101101100110111111001100101110110111010010111101110000111110101010101011 11
01101001111001010100000
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 1.95616 раз
```

Рис. 8 – Пример №4 (продолжение)

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 9.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)        (t (+ (fibonacci
 (- n 1)) (fibonacci (- n 2))))))(format t "Числа
Частоты букв:
Ч : 1
а : 1
и : 1
л : 1
с : 1
  : 38
" : 1
( : 14
) : 13
+ : 1
- : 2
0 : 2
1 : 3
2 : 1
= : 2
a : 4
b : 3
c : 7
d : 2
e : 1
f : 5
i : 6
m : 1
n : 10
o : 5
r : 1
t : 3
u : 1
```

Рис. 9 – Пример №5

```
Код букв:
Ч : 1000110
а : 0111110
и : 1000111
л : 0111001
с : 0111111
  : 11
" : 0110110
( : 010
) : 001
+ : 1000001
- : 011110
0 : 011010
1 : 00000
2 : 0110111
= : 100010
a : 10010
b : 01100
c : 0001
d : 100001
e : 0111000
f : 10011
i : 10111
m : 1000000
n : 1010
o : 10110
r : 0111011
t : 00001
u : 0111010

Закодированный текст по Хаффману: 0101000010111000100110111010101011100111011101100101101010100100001
0001101111101010100011111010000110110101010000111010010100010111010110110100011101101000111111111111
1111010010100010111010110000000111000000001111111111111111110100000111010100000111010100111011101100101
1010101001000010001101111101001111011101011000000001001110101001110111011001011010101001000010001101111
1101001111011101011011011100100100100100100100101001110110011101110000000100100000111000011101101101000
1101000111011111110111001011111 0
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 2.08765 раз
```

Рис. 10 – Пример №5 (продолжение)

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 11.

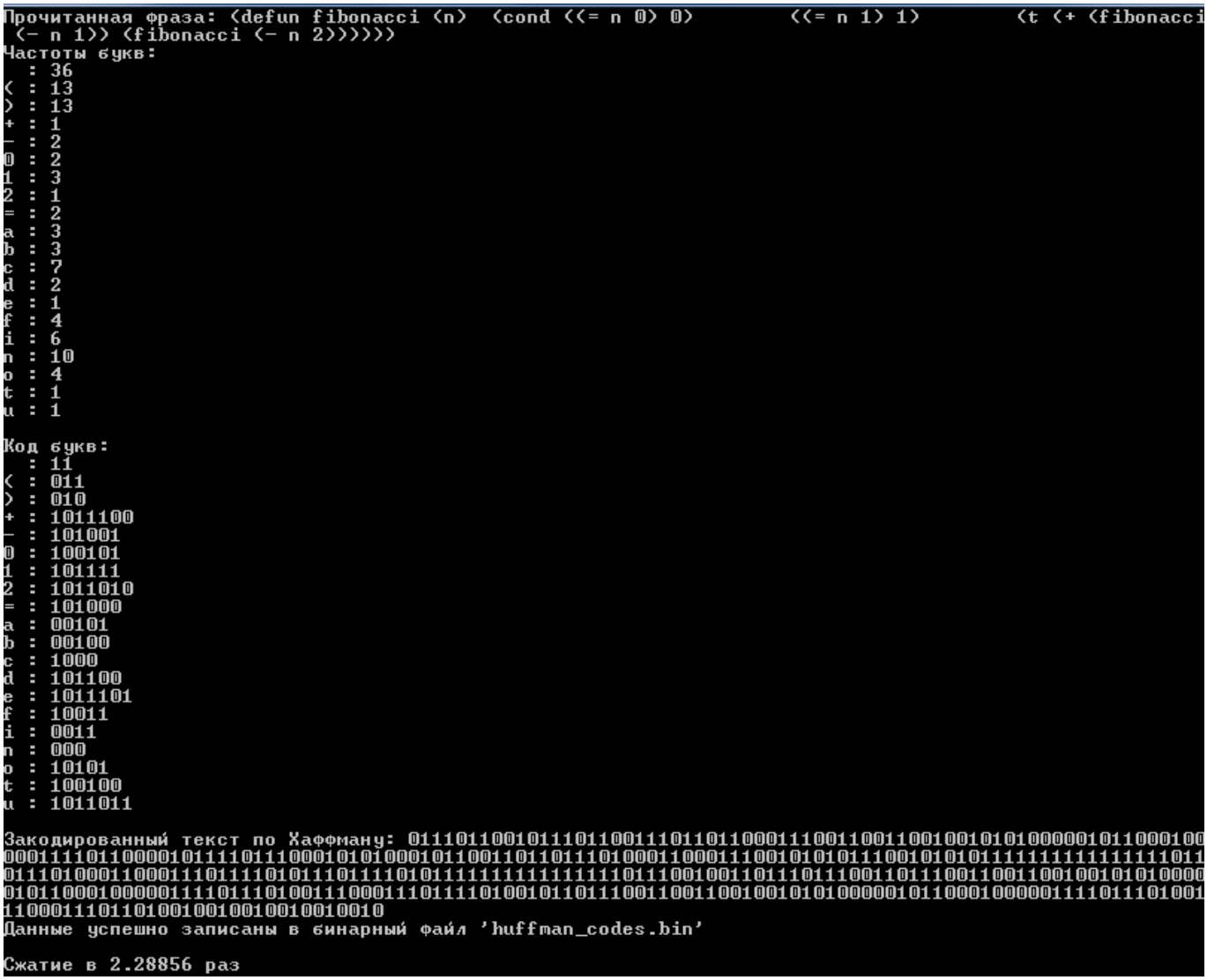

Рис. 11 – Пример №6

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 12.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)        (t (+ (fibonacci
 (- n 1))
Частоты букв:
  : 32
( : 11
) : 7
+ : 1
- : 1
0 : 2
1 : 3
= : 2
a : 2
b : 2
c : 5
d : 2
e : 1
f : 3
i : 4
n : 8
o : 3
t : 1
u : 1

Код букв:
  : 11
( : 100
) : 1011
+ : 010000
- : 010010
0 : 101011
1 : 01110
= : 00111
a : 00101
b : 00100
c : 0110
d : 00110
e : 010001
f : 01111
i : 0101
n : 000
o : 10100
t : 010011
u : 101010

Закодированный текст по Хаффману: 1000011001000010111110101000011011110101001001010000000010101100110011
0111100000010111111100011010100000000110111001000011111100011101011101111101011101111111111111111111110010
0001111110001101110101111011101011111111111111111111000100111110001000011100011110101001001010000000101
0110011001011110001001011000110111010111011
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 2.33333 раз
```

Рис. 12 – Пример №7

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 13.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)        ((= n 1) 1)
Частоты букв:
  : 19
( : 7
) : 5
0 : 2
1 : 2
= : 2
a : 1
b : 1
c : 3
d : 2
e : 1
f : 2
i : 2
n : 6
o : 2
u : 1

Код букв:
  : 11
( : 011
) : 001
0 : 10000
1 : 10111
= : 10001
a : 101100
b : 101101
c : 0001
d : 10101
e : 100100
f : 10011
i : 10100
n : 010
o : 0000
u : 100101

Закодированный текст по Хаффману: 0111010110010010011100101010111001110100101101000001010110000010001
101001101101000111110110001000001010101110110111001110101110000001111000000111111111111111111101101110
001110101110111001111011111001
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 2.36735 раз
```

Рис. 13 – Пример №8

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 14.

```
Прочитанная фраза: (defun fibonacci (n)  (cond ((= n 0) 0)
Частоты букв:
  : 8
( : 5
) : 3
0 : 2
= : 1
a : 1
b : 1
c : 3
d : 2
e : 1
f : 2
i : 2
n : 5
o : 2
u : 1

Код букв:
  : 01
( : 100
) : 1101
0 : 0001
= : 111100
a : 111101
b : 111110
c : 1110
d : 0011
e : 11000
f : 0010
i : 0000
n : 101
o : 11001
u : 111111

Закодированный текст по Хаффману: 1000011110000010111111110101001000001111101100110111110111101110000
0110010111010101100111011001101001101100100111100011010100011101010001101
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 2.21277 раз
```

Рис. 14 – Пример №9

Результат работы программы, где из файла считывается программа на LISP, сжимает данные по алгоритму Хаффмана и записывает полученный ответ в бинарный файл. Результат приведён на рис. 15.

```
Прочитанная фраза: (defun fibonacci (n)
Частоты букв:
  : 2
( : 2
) : 1
a : 1
b : 1
c : 2
d : 1
e : 1
f : 2
i : 2
n : 3
o : 1
u : 1

Код букв:
  : 1101
( : 001
) : 0100
a : 11110
b : 11111
c : 011
d : 0101
e : 1100
f : 100
i : 1110
n : 101
o : 0000
u : 0001

Закодированный текст по Хаффману: 00101011100100000110111011001110111110000101111100110111110110100 11
010100
Данные успешно записаны в бинарный файл 'huffman_codes.bin'

Сжатие в 2.19178 раз
```

Рис. 15 – Пример №10

Анализируя выполненные тесты, можно отметить, что при увеличении объема данных алгоритм Хаффмана становится более эффективным в сжатии по сравнению с простым методом сжатия. Это происходит благодаря созданию оптимального кода для представления данных: наиболее часто встречающиеся символы кодируются более короткими кодами, что позволяет значительно сократить общий размер данных.

Однако, когда данные становятся более случайными или каждый символ встречается с примерно одинаковой частотой, эффективность сжатия по Хаффману может уменьшаться. Это происходит из-за того, что оптимальное кодирование для равномерно распределенных данных не обеспечивает большого преимущества в сжатии.

Таким образом, при увеличении объема данных сжатие по Хаффману обычно стремится к более эффективному сжатию, но его эффективность может варьироваться в зависимости от характера данных и распределения символов в них.

Заключение

Цель лабораторной работы достигнута: реализован алгоритм сжатия по Хаффману.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описано сжатие по Хаффману.
2. Реализовано сжатие файла.
3. Реализована возможность считывать данные из файла.
4. Реализована возможность записать ответ в бинарный файл.
5. Выполнено тестирование реализации разработанного алгоритма.

Список литературы

1. Котиева, Х. М. Сжатие данных без потерь. Использование алгоритма Хаффмана / Х. М. Котиева. — Текст : непосредственный // Молодой ученый. — 2020. — № 35 (325 с.).